SENTINEL

Interview Preparation — Technical Deep Dive

Generated from the current local checkout on May 14, 2026

SENTINEL is a real-time market microstructure simulator and early-warning dashboard for liquidity shocks, institutional order patterns, and agent-based execution research.

Backend	Frontend	ML/RL + Replay
FastAPI, **asyncio** loop, **WebSocket** fanout, sandbox **API**s, event-driven order-book simulator	**Next.js** 15.5.15, **TypeScript**, **Tailwind CSS**, **Zustand**, **Recharts**, **lucide-react** dashboard controls	RandomForest fallback predictor, **PPO** market maker, **GP** policy, **ABIDES**-style sandbox, stock replay

Table of Contents

Major section page numbers are generated by **ReportLab** during the final multi-pass build.

1. Project Overview

SENTINEL stands for Smart Early-warning Network for Trading, Institutional orders, and Liquidity Events. The current codebase implements a real-time market microstructure simulator with a **FastAPI** backend and a **Next.js** dashboard. It creates a synthetic limit order book, runs multiple trading-agent archetypes, computes liquidity and large-order signals, and streams live market updates to a terminal-style frontend.

The current implementation is broader than a single start/stop simulator. The **API** also exposes a configurable sandbox, a lightweight **ABIDES**-style alternate engine, oracle and **latency** controls, simulation speed controls, and stock-history replay endpoints that convert downloaded **OHLCV** data into an oracle path. Those paths are visible in backend/src/api/main.py, backend/src/market/market_data.py, backend/src/market/abides_simulator.py, and the frontend **SandboxControlPanel**.

The problem it solves is controlled visibility into order-book stress. Instead of using live brokerage flow, SENTINEL simulates how market makers, HFT agents, institutional **TWAP** execution, retail participants, informed traders, noise flow, momentum traders, mean-reversion traders, spoofing behavior, sentiment behavior, and an optional **RL** market maker interact. That gives an interviewer a concrete system to discuss across simulation, distributed UI updates, **ML** signals, and deployment constraints.

Real-World Domain And Use Case

- Quant research: run controllable order-book simulations and inspect price, spread, depth, volatility, agent PnL, inventory, and warning levels.
- Risk monitoring: surface liquidity-health scores and large-order pattern detections before a simulated market stress becomes obvious in price.
- Education and demos: explain market microstructure mechanics without connecting to a real broker or risking live trades.
- Scenario testing: launch preset or custom sandboxes with controllable agent counts, oracle behavior, **latency**, speed, and optional **ABIDES**-style message passing.
- Offline experimentation: train **PPO**, **DQN**, or **GP** policies on simulator or **OHLCV** environments, then backtest them with repeatable scripts.

Evidence-Based Outcomes And Metrics In The Repo

- The backend loop broadcasts a market_update **WebSocket** packet after each simulation step and sleeps 0.1 seconds, which targets about 10 updates per second in the current implementation.
- The PRD states a success target of 10+ simulation steps per second on a developer laptop and **WebSocket** delivery without stalls for at least one hour.
- The frontend buffers incoming **WebSocket** updates and flushes the latest packet every 250 ms to keep rendering stable.
- The current backend tests pass with PYTHONPATH=backend: 10 passed with one dependency warning during this analysis.
- A trained **PPO** artifact exists at backend/models/ppo_market_maker.zip, but runtime **PPO** loading is disabled by default through **RL**_POLICY_ENABLED=false for deployment speed.
- Sandbox endpoints expose four presets, custom agent counts, oracle configuration, **latency** settings, speed changes, **ABIDES**-style simulation control, and stock replay through downloaded **OHLCV** history.

Current Scope Reality

- There is no **database**, ORM **schema**, migration folder, Redis queue, Kafka stream, or persistent event store in the current tree.
- There is no implemented **authentication** middleware. Security currently consists of configured **CORS** origins and environment-separated deployment settings.
- LIVE_SHADOW mode is exposed in **API** and UI, but the Kite client is a stub; in current simulator processing, non-SANDBOX mode prevents synthetic order processing rather than ingesting real market data.

- Some docs describe aspirational features such as **API**-key auth, data export, and richer Kite integration. The PDF treats those as planned or not implemented unless source code proves otherwise.

2. Full Tech Stack Breakdown

The stack below is based on source imports, requirements files, package.json, package-lock.json, **Docker**files, **GitHub Actions**, and runtime configuration. Where a package is declared but not imported in active code, that is called out explicitly.

Technology	Where It Is Used	Why This Choice Makes Sense Here
Python 3.10	Backend **Docker**file and Azure deployment runtime; all simulator, **API**, prediction, and training code.	The project needs numerical libraries, **RL** packages, simple async services, and portable deployment. **Python** is the most practical ecosystem for that mix.
FastAPI 0.104.1	backend/src/api/main.py exposes **REST** endpoints and **WebSocket** endpoint.	**FastAPI** gives typed request handling, Open**API** docs, **WebSockets**, and async integration without heavy framework overhead.
Uvicorn and websockets	Local backend run, **Docker** CMD, Azure startup command, and **WebSocket** serving.	The simulator is a long-running process, so an ASGI server is a natural fit.
Pydantic	ModeRequest validates the /api/simulation/mode body.	The current **API** is small, so **Pydantic** is used minimally for request **schema** validation.
python-dotenv	backend/src/utils/config.py loads backend .env values.	Keeps local and deployment configuration out of source while preserving sane defaults.
NumPy and **Pandas**	Feature extraction, **RL** observations, backtesting, metrics, training data preparation, and plotting.	They are standard for vectorized numerical work and time-series data manipulation.
yfinance / **Yahoo Finance**	backend/src/market/market_data.py lazily imports **yfinance** for stock history fetch and replay endpoints.	It gives an easy public **OHLCV** source for demos. It is optional in code and should be added to deployment requirements if stock replay is expected to work in production.
Internal **ABIDES**-style modules	backend/src/market/abides_simulator.py and backend/src/market/abides/* implement exchange, message, oracle, **latency**, and agent primitives.	The code borrows the **ABIDES** architecture idea without depending on an external **ABIDES** package, giving a second simulator mode for message-driven market experiments.
scikit-learn RandomForestClassifier	**LiquidityShockPredictor** can train/load a RandomForest model for shock classification.	Random forests are interpretable enough for tabular microstructure features and easy to fall back from.
XGBoost	Declared in root requirements.txt, but not imported by current source.	Likely reserved for alternative tabular prediction experiments; current runtime does not depend on it.
Gymnasium	**MarketMakerEnv** and **IntradayTradingEnv** implement **RL** environments.	It gives a standard reset/step/action-space contract compatible with **Stable-Baselines3**.
Stable-Baselines3, **PyTorch**, **TensorBoard**	**PPO**/**DQN** training scripts, runtime **PPO** loading, checkpoints, evaluation callbacks, and training telemetry.	**Stable-Baselines3** is a proven **RL** implementation, reducing risk versus hand-writing **PPO**/**DQN**.
Matplotlib	Visualization helpers and figures/*.py scripts.	It is used for static project/report figures and offline simulation charts.
pytest and httpx	backend/tests validate **API** contracts, **GP** policy, **RL** integration, and **RL** policy controller behavior.	They provide fast regression tests for backend behavior without launching an external server.

Frontend / Deployment Technology	Where It Is Used	Why This Choice Makes Sense Here
Next.js 15.5.15 with **React** 18	frontend/app dashboard route, layout, redirect from / to /dashboard.	**Next.js** gives App Router structure, production build support, and an easy **Vercel** deployment path.
TypeScript	Frontend types for MarketUpdate, agent metrics, dashboard state, and component props.	The **WebSocket** payload is structured, so **TypeScript** catches UI contract drift early.
Tailwind CSS ^3.4.1, resolved 3.4.19, and custom CSS variables	Terminal dashboard styling, grid layout, color system, scrollbars, panel classes.	The UI is dense and data-heavy; utility classes keep layout iteration fast while CSS variables centralize the palette.

Frontend / Deployment Technology	Where It Is Used	Why This Choice Makes Sense Here
Zustand	frontend/store/market-store.ts stores marketData, connection state, price history, alerts, simulation state, and mode.	A small client store avoids overengineering for a single-page live dashboard.
Recharts	PriceChart, PriceSpreadChart, SeriesChartPanel, TradeFlowChart.	It handles responsive charting quickly enough for browser-side visualization.
lucide-react	**SandboxControlPanel** imports Play, SlidersHorizontal, and Square icons for launch, settings, and stop controls.	Icons keep dense simulator controls recognizable without adding explanatory UI copy.
Docker Compose	docker-compose.yml runs backend on 8000 and frontend on 3000 with env files.	Useful for local full-stack smoke tests while preserving Azure/**Vercel** as the intended hosted topology.
Azure App Service	**GitHub Actions** zip workflow deploys backend and starts **Uvicorn**.	The backend must keep **in-memory** state and **WebSocket** clients, so a long-running app service is a better fit than serverless functions.
Vercel	Documented frontend hosting target with NEXT_PUBLIC_**API**_U**RL** and NEXT_PUBLIC_WS_U**RL**.	The frontend is a static/client-heavy **Next.js** app and fits **Vercel** well.
GitHub Actions	.github/workflows/main_sentinel.yml builds/tests backend, zips backend, authenticates to Azure, and deploys.	Keeps deployment repeatable and runs backend tests before publishing.

3. System Architecture

Top-Level Components

Component	Concrete Files	Responsibility
Frontend dashboard	frontend/app/dashboard/page.tsx plus components/*	Controls simulation, renders live market panels, charts, order book, agent metrics, alerts, and terminal event tape.
API service	backend/src/api/main.py	Owns global simulator, predictors, **WebSocket** manager, **REST** routes, and background simulation task.
WebSocket manager	backend/src/api/websocket.py	Accepts clients, tracks active connections, broadcasts JSON updates, and drops broken sockets.
Simulation engine	backend/src/market/simulator.py and kernel.py	Schedules agent wakeups/order arrivals, processes orders, updates state, tracks history, and returns snapshots.
Matching engine	backend/src/market/order_book.py	Maintains sorted bid/ask lists, matches market and crossing limit orders, creates Trade records, and supports cancel by order id.
Agent system	backend/src/agents/*.py	Implements strategy-specific decide_action methods and shared position/PnL accounting.
Sandbox controls	backend/src/api/main.py and frontend/components/dashboard/**SandboxControlPanel**.tsx	Exposes preset/custom agent mixes, oracle choice, **latency** mode, speed controls, stock replay, and engine selection.
ABIDES-style engine	backend/src/market/abides_simulator.py and backend/src/market/abides/*	Runs a lightweight exchange/message/oracle/**latency** simulation path alongside the native SENTINEL engine.
Market data adapter	backend/src/market/market_data.py	Fetches **OHLCV** history through **yfinance** when installed and converts it to oracle replay data.
Prediction layer	backend/src/prediction/*.py	Computes liquidity shock probability/health score and detects institutional large-order patterns.
RL layers	backend/src/market/rl_*.py and backend/src/prediction/intraday_rl/*.py	Provide runtime market-maker policy control plus offline intraday policy training/backtesting.

Full Runtime Data Flow

- A user opens /dashboard. The browser reads runtime config to infer the backend HTTP and **WebSocket** origins.
- The dashboard calls GET /api/health to synchronize simulation_active and mode, loads sandbox presets/capabilities for the control panel, then opens ws://.../ws.
- When the user clicks START SIM, POST /api/simulation/start resets predictors, optionally reloads an **RL** policy, constructs the heterogeneous agent population, creates **MarketSimulator**, and starts _run_simulation_loop with **asyncio**.create_task.
- When the user launches the sandbox path, POST /api/sandbox/create builds a **MarketSimulator** from preset or custom counts, oracle parameters, **latency** model, and speed. POST /api/sandbox/abides/create instead starts the **ABIDES**-style engine.
- When the user chooses stock replay, POST /api/sandbox/stock/fetch downloads **OHLCV** history if **yfinance** is installed, and POST /api/sandbox/stock/replay turns close prices into a replay oracle path.
- Each loop iteration optionally asks **RL**PolicyController to compute an action for **RLAgent**, then **MarketSimulator**.step advances one simulated second.
- The simulator asks externally controlled agents directly, runs scheduled events until the new target time, processes orders through **OrderBook**, updates fills and agent PnL through delayed MARKET_DATA events, replenishes thin books with a liquidity floor, and returns a market-state dictionary.
- **LiquidityShockPredictor** converts the market state into six features and returns probability, health_score, warning_level, feature values, and timestamp.

- **LargeOrderDetector** infers institutional slices from position deltas, checks **iceberg**/**TWAP** timing and size regularity, and attaches impact estimates when a pattern is found.
- The **API** builds a market_update packet containing price, spread, depth, top 10 book levels, prediction outputs, agent metrics, step, volatility, and mode.
- **ConnectionManager** broadcasts the packet to every active **WebSocket** client. The frontend stores only the latest packet and flushes to **Zustand** every 250 ms.
- The UI derives charts, alerts, depth panels, agent tables, and status cells from **Zustand** state; **REST** endpoints remain available for snapshot and prediction reads.
- The **ABIDES**-style loop emits abides_update packets with price, spread, depth, trades, message counts, agent metrics, speed, and oracle metadata; the frontend normalizes those packets into the same market display shape where possible.

API Design And Payloads

Endpoint	Request	Response / Behavior
GET /api/health	None	status, simulation_active, connected_clients, mode, rl_policy_ready, rl_policy_kind.
POST /api/simulation/start	None	Starts if not already running; returns status, agent count, initial_price, and rl_policy_active.
POST /api/simulation/stop	None	Stops simulator, cancels background task handle, resets large-order detector; returns stopped.
POST /api/simulation/mode	JSON body: { mode: SANDBOX | LIVE_SHADOW }	Validates mode with **Pydantic** body model plus explicit allowed-value check. Returns mode_updated or HTTP 400.
GET /api/prediction/liquidity	Requires active simulator	Returns probability, health_score, warning_level, features, timestamp. Returns HTTP 409 when no active simulation exists.
GET /api/prediction/large-order	Requires active simulator	Returns **iceberg**/**TWAP** detection with impact, or pattern null message. Returns HTTP 409 without simulation.
GET /api/agents/metrics	Requires active simulator	Returns per-agent total_pnl, realized_pnl, unrealized_pnl, sharpe_ratio, agent_type, position, num_trades.
GET /api/market/snapshot	Requires active simulator	Returns price, mid_price, spread, best bid/ask, depth, top bid/ask levels, volatility, and step.
GET /api/sandbox/presets	None	Returns minimal, balanced, institutional, and stress_test preset definitions with agents, oracle, **latency**, and duration.
GET /api/sandbox/capabilities	None	Returns available agent types, oracle types, **latency** models, speed range, and **ABIDES** availability flag.
POST /api/sandbox/create	SandboxCreateRequest with preset/custom agents, oracle, **latency**, speed, and duration	Stops existing simulation, builds the native simulator from requested scenario, starts the **WebSocket** loop, and returns sandbox metadata.
POST /api/sandbox/abides/create	AbidesSandboxCreateRequest	Starts the **ABIDES**-style simulator if available and returns running status, agent count, initial price, oracle type, and **latency** model.
POST/GET/PUT /api/sandbox/abides/*	Stop/status/speed requests	Stops **ABIDES** simulation, reports current status, or updates **ABIDES** speed multiplier.
PUT /api/sandbox/speed	SpeedRequest	Updates native simulator speed multiplier within configured bounds.
GET /api/sandbox/oracle	Requires active simulator with oracle	Returns oracle type, current fundamental value, volatility, and mean-reversion settings when present.
GET /api/sandbox/stocks/popular	None	Returns curated ticker symbols such as AAPL, TSLA, NVDA, SPY, BTC-USD, ^NSEI, and ^BSESN.
POST /api/sandbox/stock/fetch	StockFetchRequest	Fetches historical **OHLCV** data through **yfinance** when installed and returns metadata and recent rows.
POST /api/sandbox/stock/replay	StockReplayRequest	Creates a native replay simulation from historical close prices and starts streaming the replay through the normal market_update path.
WS /ws	Client connects and may send keepalive messages	Server broadcasts market_update JSON packets and disconnects failed clients.

```
market_update = {
  "type": "market_update",
  "timestamp": current_time,
  "price": current_price,
  "spread": spread,
  "depth": total_depth,
  "order_book": {"bids": top_10_bids, "asks": top_10_asks},
  "liquidity_prediction": {"probability", "health_score", "warning_level", "features", "timestamp"},
  "large_order_detection": null | {"pattern", "side", "estimated_size", "confidence", "impact"},
  "agent_metrics": {"AGENT_ID": {"total_pnl", "realized_pnl", "unrealized_pnl", "sharpe_ratio", "agent_type", "positi
  "step": step_count,
  "volatility": annualized_volatility,
```

```
  "mode": "SANDBOX" | "LIVE_SHADOW"
}
```

Data Model, Storage, And Database Reality

There is no relational or document **database** in the current codebase, and no **schema** or migration files were found. The runtime data model is made of **Python** dataclasses, dictionaries, deques, **in-memory** lists, **TypeScript** interfaces, and optional file artifacts such as model zip/json files, CSV training data, JSON checkpoint metadata, and generated results.

Entity / Interface	Fields / Relationship	Storage
Order	agent_id, side, order_type, price, quantity, order_id, timestamp, filled_quantity, status. Belongs to an agent and may rest in bid/ask lists.	In-memory dataclass.
Trade	buyer/seller order ids, buyer/seller agent ids, price, quantity, trade id, timestamp. Created by **OrderBook** when two orders match.	In-memory dataclass list.
Agent	position, avg_entry_price, realized_pnl, active_orders, strategy parameters. Agents produce orders and receive fills.	In-memory object per simulation.
Market state	current_time, mid, best bid/ask, spread, depth, imbalance, signed volume, fill rate, volatility, agents, step.	Computed dictionary snapshot.
Sandbox request	preset_name or custom_agents, initial_price, duration, oracle_type, mean_reversion, fundamental_value, **latency**_model, speed_multiplier.	**Pydantic** request body, then **in-memory** simulator configuration.
Oracle and **latency**	**MeanRevertingOracle**/replay oracle values plus **LatencyModel** settings influence fundamental values and event timing.	In-memory simulation dependencies.
ABIDES message state	Exchange, market maker, noise, informed agents, order messages, trades, and message counts are held by **AbidesSimulation**.	Separate **in-memory** simulator path.
MarketUpdate	**TypeScript** interface mirroring the **WebSocket** packet and prediction/agent contracts.	Browser memory through **Zustand**.
Checkpoint metadata	checkpoint_metadata.json, best_models.json, sweep_checkpoint.json for offline **RL** workflows.	Local JSON files under backend/checkpoints.

Async, Caching, And Event-Driven Patterns

- Backend **concurrency** uses **asyncio** for a single background simulation task and async **FastAPI/WebSocket** handlers.
- Market scheduling uses a heap-backed **priority queue** in **EventKernel**; this is an internal event queue, not an external broker.
- The **ABIDES**-style path uses a separate event/message loop, exchange agent, oracle, and **latency** model, but it is still process-local rather than an external event bus.
- **WebSocket** broadcast is sequential over active connections and removes failed sockets after send exceptions.
- Frontend **WebSocket** code implements exponential reconnect backoff up to five retries.
- Frontend update throttling stores the latest packet in a ref and flushes it every 250 ms; this is a render-stability optimization rather than server-side caching.
- The **API** keeps simulator state as global process memory, so multi-instance scaling would require external state and **WebSocket** fanout.

4. Key Technical Decisions

Decision	Reasoning Inferred From The Code	Trade-Off
Use **FastAPI** with a long-running process	The simulator needs background tasks, process-local state, **WebSockets**, and health/control **API**s.	Great for a single stateful service; not a fit for stateless serverless scale without redesign.
Use **WebSocket** for live market updates and **REST** for control/read endpoints	Control actions are discrete, while market state is continuous and push-based.	Clean separation, but **WebSocket** fanout needs more infrastructure for many clients.
Keep simulator state in memory	The dashboard/demo needs fast state mutation and no persistence setup.	Simple and low-**latency**, but restarts wipe state and multi-replica deployment is unsafe.
Use an **EventKernel priority queue**	Agent wakeups, order arrivals, cancels, and market-data updates need deterministic temporal ordering.	More realistic than a flat loop, but still single-process and not distributed.
Use simple sorted **Python** lists for bids and asks	The book is small enough for demos/tests, and price-time logic is readable.	Insertion is O(n), so high-volume production order books need trees/heaps/price-level maps.
Model agents as subclasses with decide_action	Every participant shares accounting but differs in strategy.	Easy to extend, but strategy coupling to market_state dictionary can drift without broader tests.
Expose sandbox presets plus custom counts	The dashboard can demonstrate different market regimes without code changes or manual config files.	More request validation and UI state are needed, and unrealistic presets can produce misleading conclusions.
Add a lightweight **ABIDES**-style engine	**ABIDES** concepts map well to exchange/message/oracle/**latency** experiments, but a full external **ABIDES** dependency would be heavier than the demo needs.	The repo now has two simulator paths, so parity and status synchronization need careful tests.
Use **yfinance** lazily for stock replay	Stock replay is useful for demos, but the base backend should still start when **yfinance** is absent.	Replay endpoints fail unless **yfinance** is installed and network access works; deployment requirements need to reflect that if the feature is promised.
Disable **PPO** runtime loading by default	**Stable-Baselines3** is heavier than the base backend deployment path, and loading a model slows health/startup.	Fast deployments are easier; live **RL** requires explicit dependency/config setup.
Provide heuristic fallback for liquidity prediction	A demo should work without a trained liquidity_model.pkl.	Signals are explainable and robust, but less statistically calibrated without training data.
Separate intraday **RL** from live simulator runtime	**OHLCV** policy training has a different data model and evaluation contract than the simulated limit-order-book environment.	This keeps experiments flexible, but the repo now has two **RL** systems to explain clearly.
Use **Vercel** frontend plus Azure backend	The frontend is static/client-heavy; the backend is stateful and **WebSocket**-based.	Matches platform strengths, but introduces cross-origin and environment-variable setup.
Expose LIVE_SHADOW now but keep Kite as a stub	The UI/**API** can show future mode semantics while avoiding brokerage credentials and live-data complexity.	Interviewers may ask why the mode exists before full implementation; answer honestly that it is a placeholder path.
Avoid **database** in current version	Simulation and dashboard state can be computed in memory for the current demo scope.	Simplifies setup, but prevents audit logs, replay, analytics history, and horizontal scaling.

Auth, Validation, And Error Handling Patterns

- Authentication is not implemented in the active **FastAPI** app. **API**-key auth appears only in aspirational docs.
- **CORS** is configured from FRONTEND_U**RL** and ALLOWED_ORIGINS, with localhost defaults and de-duplication.

- ModeRequest, SandboxCreateRequest, AbidesSandboxCreateRequest, SpeedRequest, StockFetchRequest, and StockReplayRequest are **Pydantic** BaseModel request contracts.
- The mode endpoint enforces SANDBOX or LIVE_SHADOW and returns HTTP 400 on invalid mode. Speed endpoints enforce configured ranges before mutating simulator speed.
- Prediction, metrics, and snapshot endpoints call _require_simulator and return HTTP 409 when no active simulation exists.
- Stock fetch/replay endpoints convert provider errors into HTTP 500 responses with explicit error text, while replay validates that enough price data exists before starting a simulation.
- Order.fill raises ValueError for non-positive or over-remaining fills; simulator rejects non-positive quantities and non-finite prices.
- **RLAgent** sanitizes non-finite actions and clips normalized action components to [-1, 1]. **RL**PolicyController falls back to neutral actions for non-finite policy output.
- Intraday feature loading validates timestamp/DatetimeIndex and **OHLCV** columns. Intraday environment validates sessions, action ids, and session indexes.
- Frontend **API** client throws on non-OK HTTP responses. **WebSocket** parsing ignores malformed messages and reconnects after close.

5. Deep Dive - Hardest Problems

1. Correct Temporal Ordering In A Simulated Market

The hardest backend problem is keeping market events ordered while many agents behave at different frequencies and latencies. **MarketSimulator** sorts agents by **latency**, schedules first wakeups, then uses **EventKernel** to process WAKEUP, ORDER_ARRIVAL, CANCEL_ARRIVAL, and MARKET_DATA events in timestamp order. That lets HFT, market makers, retail, institutional, and slower behavioral agents share one market without all acting at the same artificial moment.

2. Order-Book Correctness

OrderBook has to maintain **price-time priority**, partial fills, market-order cancellation of unfilled remainders, resting limit orders, and agent-side active order cleanup. Equal-price time priority is preserved by appending behind existing orders at the same price. The simulator also schedules stale-order cancellation through the event kernel so market makers and other quoting agents can replace old quotes.

3. Stateful Async Runtime Without Locks

The **API** uses a global simulator and a single background **asyncio** task. That is pragmatic for a demo, but it means concurrent **REST** calls, the simulation loop, and **WebSocket** clients all observe the same mutable object. The code reduces complexity by running state mutation in the single event-loop process, but it would need explicit locking or state externalization for multi-worker production.

4. Latency And UI Stability

The backend targets a live feed, but the frontend avoids rerendering on every raw message. useMarket**WebSocket** keeps the newest packet in a ref and flushes at 250 ms. This means the browser can display a stable terminal dashboard even if the backend sends about 10 updates per second.

5. Safe RL Policy Integration

The **RL** market maker is externally action-controlled, but still routes quotes through the same simulator order path as every other agent. **RL**PolicyController extracts the 13-feature observation, normalizes if **VecNormalize** is present, predicts with **PPO** or **GP**, clips non-finite output, and queues the action on **RLAgent**. **RLAgent** then maps normalized spread/skew/size into actual quotes and applies inventory, volatility, flow, time-to-close, and quote-order safeguards.

6. Large-Order Detection Without A Real Trade Tape

LargeOrderDetector does not receive raw institutional parent orders. It infers executed slices from changes in institutional agent inventory. From there it detects **iceberg**-like regular sizes and **TWAP**-like regular timing using coefficients of variation, then estimates impact from estimated size versus displayed depth and volatility. That is a realistic compromise for a simulator where the hidden parent order is not directly exposed.

7. Keeping Multiple Simulation Modes Coherent

The code now has the default SENTINEL simulator, configurable sandbox launches, stock replay using a replay oracle, and an **ABIDES**-style alternate engine. The non-obvious challenge is presenting those through one dashboard without pretending they are identical. The native simulator emits market_update packets with prediction and agent metrics, while the **ABIDES**-style path emits abides_update packets with message counts and exchange-oriented state. The frontend normalizes enough fields for a unified display while still preserving engine-specific controls and status.

8. Scale Constraints To Explain Clearly

- The sorted-list **OrderBook** is readable but not optimized for millions of orders.
- Sequential **WebSocket** broadcast is fine for a few clients but needs fanout/backpressure for many clients.
- In-memory state prevents multi-instance backend scaling.
- No persistent event log means replay, audit, and post-run analytics are limited.

- **yfinance**-backed replay depends on optional dependency installation, external network access, and provider availability.
- LIVE_SHADOW needs real Kite integration before it can be described as live market ingestion.

6. ML/AI Specifics

Liquidity Shock Model

LiquidityShockPredictor is a tabular **ML** component. It tries to load a pickled **RandomForestClassifier** from backend/models/liquidity_model.pkl. If no model exists, it uses a deterministic heuristic so the **API** still returns useful output.

Aspect	Implementation Detail
Features	spread_ratio, depth_ratio, volatility_ratio, mm_inventory_stress, active_mm_count, time_to_close.
Training labels	generate_training_data labels a sample as shock if, within the next 60 steps, depth_ratio drops below 0.5 or spread_ratio exceeds 3.0.
Model	**RandomForestClassifier** with 100 estimators, max_depth=10, class_weight=balanced, random_state=42.
Inference	predict builds a feature row, calls predict_proba when model exists, otherwise uses heuristic scoring.
Output	probability, health_score from 0 to 100, warning_level safe/caution/warning/critical, features, timestamp.
Explainability	The feature dictionary is returned directly to the frontend, so users can see why the health score moved.

Runtime PPO / GP Market-Maker Policy

Aspect	Implementation Detail
Observation	13 normalized values: best bid, best ask, spread, mid, imbalance, recent price change, signed volume, inventory, realized PnL, unrealized PnL, volatility, fill rate, and time remaining.
Action	Continuous 3-vector in [-1, 1]: spread, skew, quote size.
PPO architecture	**Stable-Baselines3 PPO** with MlpPolicy, Tanh activation, pi/vf network [128, 128], gamma 0.995, gae_lambda 0.98, entropy 0.01, **VecNormalize** saved alongside model.
Reward shaping	Spread capture, mark-to-market PnL, two-sided quote bonus, quote quality bonus minus inventory, cancel, drawdown, adverse selection, and closeout penalties.
Inference serving	**RL**PolicyController loads **PPO** or **GP** policy, prepares action before simulator step, and **RLAgent** submits resulting quotes through the simulator.
Failure handling	Missing model or missing **Stable-Baselines3** disables runtime **RL**; non-finite observations/actions are sanitized.
Current artifact	backend/models/ppo_market_maker.zip exists and contains **Stable-Baselines3** model files, but live autoload is disabled unless **RL**_POLICY_ENABLED=true.

Genetic Programming Policy

The **GP** policy is an interpretable alternative to **PPO**. A genome contains three expression trees, one for each action dimension. Nodes can be constants, feature references, unary operations such as neg/abs/tanh, or binary operations such as add/sub/mul/div. Training uses tournament selection, mutation, crossover, elitism, and simulator-based fitness.

Intraday OHLCV PPO/DQN System

The second **RL** system under backend/src/prediction/intraday_rl is not the same as the live order-book simulator. It trains policies on minute **OHLCV** sessions. The feature layer validates timestamp/open/high/low/close/volume data, computes **SMA**, **RSI**, **ATV**/volume_ratio, returns, splits daily sessions, and can augment sessions with price and volume noise.

Intraday Area	Implementation
Environment	**IntradayTradingEnv** with discrete actions HOLD, BUY, SELL; long/flat position; first 60 minutes trading window; trailing stop; one reverse re-entry.
Observation	lookback * 5 candle features plus six extras: position, PnL, **SMA** signal, **RSI**, volume ratio, and morning progress.

Intraday Area	Implementation
Training	train_ppo and train_dqn_baseline use DummyVecEnv, **Stable-Baselines3**, optional checkpoint callback, and resumable loading.
Backtesting	backtest_model runs deterministic sessions and reports final equity, total PnL, return percent, drawdown, trade counts, win/loss counts, and Sharpe-like summary.
Orchestration	train_backtest_rl_system.py adds **CheckpointManager**, named runs, backtest metrics, resumable parameter sweep, JSON results, and CLI subcommands.
Evaluation metrics	avg_return, total_return, win_rate, sharpe, max_drawdown, profit_factor, num_trades.

ML/AI Failure Modes To Mention In Interviews

- The liquidity model falls back to heuristics if a trained pickle is absent, so probabilities are not calibrated unless the model is trained and validated.
- Simulator-trained policies may overfit synthetic agent behavior and fail on real market regimes.
- Intraday **RL** can leak information if train/test dates or rolling indicators are split incorrectly.
- Transaction costs exist in the intraday environment, but explicit slippage and market impact modeling are still improvement areas.
- Runtime **PPO** dependency weight is intentionally isolated from the default Azure backend deployment path.

7. Interview Questions and Answers

Question: 1. Give me the architecture of SENTINEL end to end.

Answer: SENTINEL has a **Next.js** dashboard, a **FastAPI** backend, an **in-memory** native market simulator, a lightweight **ABIDES**-style simulator path, a prediction layer, and optional **RL** policy control. The dashboard calls **REST** endpoints to start, stop, switch mode, create sandboxes, fetch stock replay data, adjust speed, and read snapshots, then subscribes to /ws for market_update or abides_update packets. The backend owns global singletons for simulator, **ABIDES** simulator, predictors, **RL** policy controller, and **WebSocket** manager. Starting a native simulation constructs a heterogeneous agent population, creates **MarketSimulator**, and launches an **asyncio** background loop. Each step advances the event kernel, processes orders through **OrderBook**, computes liquidity and large-order signals, then broadcasts compact JSON to the browser. The frontend stores normalized live state in **Zustand** and renders charts, alerts, order book, agent metrics, and sandbox controls.

Concept tested: System architecture and data flow.

Question: 2. Why did you choose FastAPI for the backend?

Answer: **FastAPI** fits the actual runtime shape: async **REST** endpoints, a **WebSocket** route, simple **Pydantic** validation, Open**API** docs, and **Uvicorn** deployment. The simulator is stateful and long-running, so the backend needs to manage background tasks and **WebSocket** clients in one process. A heavier framework would add unused surface area, while a serverless-only approach would not work well because the simulator and client connections live across requests.

Concept tested: Framework selection and deployment fit.

Question: 3. How does the order book maintain correctness?

Answer: **OrderBook** stores bids in descending price order and asks in ascending price order. Market orders consume the best opposite side until filled or liquidity runs out; leftover market quantity is cancelled. Limit orders first cross against eligible opposite-side prices, then rest any remaining quantity. The fill method prevents negative or over-sized fills. Equal-price orders are appended behind existing orders, preserving time priority. Trades update both orders and are later routed back to the buyer and seller agents for position and PnL updates.

Concept tested: Market microstructure **correctness**.

Question: 4. How do you model concurrency and latency?

Answer: The project uses two layers. At the web-service level, **FastAPI** and **asyncio** run **REST**/**WebSocket** handlers plus one simulation task. At the market-simulation level, **EventKernel** is a heap-backed **priority queue**. Agents wake up at different intervals and have **latency**_seconds values, so orders and market-data notifications are scheduled into the future. The simulator processes events in timestamp order up to the next simulated second. This gives deterministic event ordering without real threads per agent.

Concept tested: Concurrency model, event queues, **latency** simulation.

Question: 5. What happens when there is no active simulation and a client asks for market data?

Answer: The backend uses _require_simulator for prediction, agent metrics, and market snapshot endpoints. If simulator is None, it raises HTTPException with status 409 and detail No active simulation. That is more precise than returning empty fake data because the client called a stateful endpoint before starting the simulation. The frontend separately checks health and resets local state when simulation_active is false.

Concept tested: **API** error handling and state contracts.

Question: 6. How does the liquidity shock predictor work?

Answer: It extracts six features from current market state: spread_ratio, depth_ratio, volatility_ratio, market-maker inventory stress, active market-maker count, and time_to_close. If a pickled **RandomForestClassifier** exists, it uses predict_proba to estimate shock probability. If not, it applies a heuristic that increases probability for wide spreads, thin depth, high volatility, stressed market makers, and too few active market makers. Health score is one minus shock probability, scaled to 0-100, then mapped to safe, caution, warning, or critical.

Concept tested: **ML** feature engineering and fallback behavior.

Question: 7. How does the large-order detector identify iceberg or TWAP behavior?

Answer: The detector records institutional inventory changes as executed slices. It then checks recent buy and sell histories. Iceberg detection looks for consistent order sizes and regular timing; **TWAP** detection focuses on regular time intervals. Both use coefficient-of-variation style checks and require estimated size above the min_order_size threshold. When a pattern is found, it estimates market impact from estimated size divided by displayed depth and scaled by volatility.

Concept tested: Pattern detection, statistics, and explainability.

Question: 8. What is the RL model doing in the live simulator?

Answer: The runtime **RL** path controls an **RLAgent** that acts as a market maker. The policy sees a 13-feature observation about best prices, spread, imbalance, volume, inventory, PnL, volatility, fill rate, and time remaining. It outputs normalized spread, skew, and quote size. **RLAgent** decodes that into actual bid/ask quotes, then adjusts for current spread, volatility, inventory, signed flow, and time-to-close. The resulting quotes still go through the same simulator and order book as every other agent.

Concept tested: **RL** serving and safe action mapping.

Question: 9. Where is the database schema, and how are relationships modeled?

Answer: There is no **database schema** or migration system in the current codebase. Relationships are modeled in memory: Orders belong to agents and rest in **OrderBook** bid/ask lists; Trades link buyer and seller order ids and agent ids; **MarketSimulator** owns agents, order book, histories, and event kernel; the frontend uses **TypeScript** interfaces for the **WebSocket** contract. For production replay or analytics, I would add an append-only event log and store orders, trades, snapshots, predictions, and agent metrics in Postgres or Parquet depending on query needs.

Concept tested: Data modeling and honest scope discussion.

Question: 10. How would you scale this system?

Answer: First I would split read-only UI fanout from simulation ownership. The current backend should run as a single simulation owner because state is process-local. To scale clients, publish market_update packets to Redis Pub/Sub, NATS, or a managed stream and run separate **WebSocket** fanout workers. To scale simulations, shard by simulation id and persist state snapshots/events. For order-book performance, replace sorted lists with price-level maps and queues. For reliability, add persistence, auth, rate limits, metrics, and a replay/export **API**.

Concept tested: Scalability, state ownership, and incremental architecture.

Question: 11. What would you change if you had more time?

Answer: I would add persistent event sourcing for orders, trades, snapshots, predictions, and alerts; implement **API authentication** and rate limiting; complete LIVE_SHADOW with a real Kite provider or remove the toggle until ready; add stronger tests for the matching engine and agent lifecycle; add slippage/impact to the intraday **RL** backtester; and expose replay/export endpoints. Those changes directly address the current limits visible in the code rather than adding unrelated features.

Concept tested: Technical judgment and roadmap prioritization.

Question: 12. How do the sandbox, ABIDES-style engine, and stock replay fit into the architecture?

Answer: They are alternate ways to create a market scenario without changing the core dashboard. The normal /api/simulation/start path builds a fixed native SENTINEL agent population. /api/sandbox/create builds the same native simulator from presets or custom agent counts plus oracle, **latency**, speed, and duration settings. /api/sandbox/abides/create starts a separate **ABIDES**-style simulator built around exchange, message, oracle, **latency**, and agent primitives. /api/sandbox/stock/fetch and /api/sandbox/stock/replay use **yfinance**-provided **OHLCV** history, when installed, to build a replay oracle path. The frontend **SandboxControlPanel** chooses the engine and scenario, while the **WebSocket** hook normalizes native market_update and **ABIDES** abides_update packets enough for the dashboard to render them consistently.

Concept tested: Scenario design, alternate engine boundaries, and live-data honesty.

8. Follow-up Questions

Question: 1. Why is LIVE_SHADOW risky to describe as complete?

Answer: The UI and **API** expose LIVE_SHADOW, and **KiteClient** has a future-facing stub, but current _process_order returns immediately when mode is not SANDBOX. There is no real Kite ingestion path wired into **MarketSimulator**. The stock replay endpoints are different: they can replay downloaded historical **OHLCV** prices as an oracle path, but that is still not live brokerage ingestion. I would describe LIVE_SHADOW as a planned integration surface, not a completed live-data mode.

Concept tested: Honest product scope and code truth.

Question: 2. Why not store everything in a database from the beginning?

Answer: For this version, the priority was an interactive simulator and dashboard with minimal setup. In-memory state makes the loop simple and fast. A **database** becomes necessary when the product needs replay, audit logs, multi-user simulations, long-term analytics, or horizontal scale.

Concept tested: Trade-offs in MVP architecture.

Question: 3. How do you prevent frontend performance problems with frequent WebSocket updates?

Answer: The **WebSocket** hook does not push every message directly into **React** state. It stores the newest market_update in a ref and flushes it every 250 ms. **Zustand** keeps the shared state small and caps priceHistory at 240 points, which reduces chart and table churn.

Concept tested: Client-side performance and state management.

Question: 4. What tests are currently most valuable?

Answer: The backend tests cover **API** error contracts, **RL** quote lifecycle through the simulator, policy controller sanitization/deferred loading, **GP** model round-trip, and a small **GP** training smoke test. Those tests protect the unusual parts of the system: stateful **API**s, **RL** action routing, and policy serialization.

Concept tested: Testing strategy.

Question: 5. What is the biggest correctness gap?

Answer: The order book and simulator have tests around **RL** integration, but they need more direct matching-engine tests for partial fills, equal-price time priority, cancellation timing, invalid order rejection, and agent PnL on buy/sell flips. That would be the first **correctness** hardening step.

Concept tested: Residual risk identification.

Question: 6. How would you secure it for a hosted demo?

Answer: Add **API**-key or session auth for state-changing endpoints, keep **CORS** restricted to the **Vercel** domain, rate-limit start/stop/mode changes, separate public read endpoints from admin controls, store secrets in Azure/**Vercel** settings, and avoid exposing brokerage credentials to the browser.

Concept tested: Security design.

9. Quick Reference Card

Elevator Pitch

SENTINEL is a real-time market microstructure research system. It runs a multi-agent limit-order-book simulator in **FastAPI**, computes explainable liquidity and large-order warning signals, supports configurable sandbox, **ABIDES**-style, and stock-replay scenarios, optionally injects **RL** market-maker policies, and streams live state over **WebSockets** into a dense **Next.js** trading dashboard. The project is strongest when framed as an end-to-end simulation, **ML**/**RL**, and live-UI engineering system rather than as a real brokerage platform.

5 Most Impressive Technical Facts

- The simulator uses a priority-queue **EventKernel** so agents act with distinct **latency** and wakeup timing rather than in a flat loop.
- The matching engine implements **price-time priority**, partial fills, market-order remainder cancellation, resting order cancellation, and trade-linked agent PnL updates.
- The live **API** combines **REST** control endpoints, sandbox/replay endpoints, **WebSocket** market_update packets, **ABIDES** abides_update packets, and frontend throttling for stable real-time rendering.
- The project has multiple AI paths: RandomForest liquidity prediction, heuristic fallback, **PPO** market-maker control, **GP** policy search, and an intraday **PPO**/**DQN** training/backtesting system.
- Deployment thinking is explicit: Azure for the stateful backend, **Vercel** for the frontend, **Docker Compose** for local smoke testing, and **RL** disabled by default to keep production startup fast.

3 Things That Make It Stand Out From Typical Student Projects

- It has domain-specific core logic: order matching, agent strategies, inventory risk, **TWAP**/**iceberg** detection, liquidity scoring, oracle/replay controls, and market-state feature engineering.
- It acknowledges operational trade-offs clearly: **in-memory** state, single-instance backend, optional heavy **RL** dependencies, no **database** yet, and no implemented auth yet.
- It includes verification and deployment assets, not just app code: pytest tests, **Docker**files, **Docker Compose**, Azure **GitHub Actions**, environment examples, and training/backtesting scripts.

Commands To Remember

```
# Backend tests from repo root in PowerShell
$env:PYTHONPATH='backend'; python -m pytest -q backend/tests

# Backend dev server
cd backend
python -m uvicorn src.api.main:app --reload --host 0.0.0.0 --port 8000

# Frontend
cd frontend
npm run lint
npm run build
npm run dev

# Full-stack local smoke
docker compose up --build

# Runtime market-making PPO
python train_rl.py --timesteps 60000 --save-path backend/models/ppo_market_maker

# GP market-making policy
python train_gp.py --generations 8 --population-size 24 --save-path backend/models/gp_market_maker.json
```